TECHNICAL REPORT

Rust API Project

Database, API, and Infrastructure

Author: Lucas Fagioli
Date: 24 March 2026
Repository: `rust-api`
Version: 0.1.0
Scope: Schema, backend architecture, benchmarks, deployment, and observability

Executive Summary

This document provides a structured technical reading of the Rust API repository. It studies the PostgreSQL schema table by table, explains the internal organization of the Rust codebase and the HTTP surface, documents the self-hosted deployment model built around Docker, PostgreSQL, Redis, Nginx, `pass`, and `cryptsetup`, and describes the monitoring stack used to observe logs and security-relevant runtime behavior. The report is intended to serve as a durable engineering reference for maintenance, onboarding, performance work, release operations, and production observation.

Internal technical documentation
March 2026

Contents

I.

Introduction

1 Project Context and Objectives

The Rust API repository already contains the main elements of a production-oriented backend: a non-trivial PostgreSQL schema, layered Rust application code, a real HTTP surface, two Docker images, deployment documentation, and CI workflows. That makes it possible to write a report that is both descriptive and analytical. A short repository description would not be enough here, because the project already has architectural choices that need to be explained in context: why sessions are stored durably, why tokens are hashed at rest, why audit logs are partitioned, why Redis is separated from PostgreSQL, and why migrations are published as their own image rather than being treated as a side effect of the runtime container.

This report therefore has five main objectives. First, it documents the shape of the schema, table by table, so that future maintenance does not rely exclusively on reading migrations in chronological order. Second, it explains how the Rust codebase is organized and how HTTP requests move from router to service to repository. Third, it records the current deployment model as a deliberate architecture rather than a set of ad hoc shell commands. Fourth, it describes the monitoring model adopted around Grafana, Loki, Alloy, and read-only SQL observation. Fifth, it identifies the main operational and performance characteristics of the project, with special attention to the distinction between cryptographic cost and database cost.

1.1 Method and Limits

This report is written from the repository itself, not from a live production deployment. That distinction matters. The schema, routes, Docker assets, deployment guides, and benchmark harnesses are all available in the repository and can therefore be described directly. In contrast, exact relation sizes, exact table growth curves, and benchmark numbers for a specific machine are not embedded in the repository. For that reason:

- storage figures in the database chapter are engineering estimates derived from column types and typical payload lengths;
- operational size formulas are presented explicitly as models, not as measured production facts;
- the performance chapter focuses on benchmark coverage, dominant cost centers, and latency classes rather than pretending that one local benchmark run is universally authoritative.

This methodological limitation is not a weakness. On the contrary, it makes the report more honest and more maintainable. The document provides the framework used to reason about the project, and production measurements can later be inserted into that framework without having to redesign the report itself.

II.

Database

2 Database Objectives and Design Principles

PostgreSQL is the durable source of truth of the project. Every security-sensitive state transition that must survive a process restart eventually lands in PostgreSQL: user lifecycle state, role assignment, second-factor enrollment, refresh-token lineage, one-time verification or reset material, risk telemetry, and audit events. In a system of this kind, the schema is not merely a storage detail. It directly participates in the security model.

Three design principles are particularly visible in the migrations.

2.1 Security State Is Stored Durably

The project does not reduce authentication to "accept a JWT and trust it until it expires." Instead, the runtime API ties JWTs and refresh flows back to persistent session state. This allows explicit revocation, compromise marking, refresh-token reuse detection, and family-wide invalidation. In other words, session state remains visible and mutable after token issuance, which is a stronger model than pure stateless token handling.

2.2 One-Time Material Is Hashed at Rest

The repository stores refresh tokens, email verification tokens, password reset tokens, email 2FA codes, and recovery codes as hashes when plaintext recovery is not needed. This is an important design choice. If the database were disclosed, those values would not automatically become replayable credentials. The schema therefore reflects the same principle as password hashing: avoid storing secrets in recoverable form unless recovery is functionally required.

2.3 Consistency Is Enforced in SQL, Not Only in Rust

The migrations define many constraints that prevent invalid states before the application code even sees them. Examples include:

- username and locale format checks;
- email verification consistency with user status;
- session replacement and compromise metadata consistency;

- second-factor payload constraints by method type;
- one-active-token semantics for verification and reset flows;
- append-only audit log enforcement.

This is a healthy pattern. The Rust application still performs validation, but the schema acts as the final line of defense for invariants that should never be broken.

3 Schema Map

The schema can be read as three large areas: identity and authorization, session and authentication state, and operational telemetry. Table 1 gives a high-level view before the detailed, table-by-table analysis.

Table 1: Logical grouping of the main relations

Group	Main relations	Role in the system
Identity and authorization	`users`, `roles`, `permissions`, `role_permissions`, `user_roles`	Represents accounts, lifecycle state, default roles, and effective authorization.
Session and authentication state	`sessions`, `two_factor_methods`, `email_2fa_codes`, `email_verification_tokens`, `password_reset_tokens`, `recovery_codes`	Persists the security artifacts needed to authenticate, verify, challenge, rotate, revoke, and recover access.
Operational telemetry	`login_attempts`, `audit_log`, `login_locations`	Records behavior, suspicious activity, operational history, and long-lived audit evidence.

4 Table-by-Table Analysis

This section is intentionally detailed. Each subsection explains what the table contributes to the overall security model, then gives a field-by-field size estimate. The figures are not exact PostgreSQL heap measurements. They are typical logical payload estimates that help with capacity planning and with identifying which relations are naturally light and which ones can become heavy over time.

4.1 Identity and Authorization

4.1.1 `users`

The `users` table is the anchor of the entire schema. It is not limited to a username and a password hash. It also captures account lifecycle state, locale, verification state, lockout state, and last login timestamp. This is the correct shape for a security-oriented user table because most important decisions during login and profile updates depend on a coherent view of the account as a whole.

Table 2: `users`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key generated by `gen_random_uuid()`.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`updated_at`	`TIMESTAMPTZ`	8 B	Maintained by the generic update trigger.
`email_verified_at`	`TIMESTAMPTZ`	8 B	Present only once verification succeeds.
`last_login_at`	`TIMESTAMPTZ`	8 B	Updated on successful authentication.
`locked_until`	`TIMESTAMPTZ`	8 B	Used by temporary lockout logic.
`status`	`user_status`	4 B	Enum for active, inactive, suspended, or pending verification.
`preferred_locale`	`VARCHAR(10)`	2–10 B	Usually a short value such as `en` or `en_US`.
`username`	`VARCHAR(50)`	8–24 B	Short user-controlled identifier.
`email`	`CITEXT`	20–40 B	Case-insensitive login identifier.
`password_hash`	`TEXT`	95–110 B	Argon2id encoded hash string.
Estimated logical row size	**approx. 185–240 B**		Planning model: `row_count x 185-240 B` for heap payload, plus unique indexes on username and email and secondary lifecycle indexes.

For sizing discussions, a practical planning rule is to think of `users` as a relation whose business payload is modest but whose indexes matter a lot. The table itself remains relatively small in most deployments; the more important cost is the set of unique and secondary indexes used by login and lifecycle queries.

4.1.2 `roles`

The `roles` table is intentionally small and stable. Its main job is to provide named authorization bundles, plus a single "default role" concept for new registrations. This is a low-growth table whose importance lies in semantic clarity rather than storage weight.

Table 3: `roles`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`is_default`	`BOOLEAN`	1 B	Marks the unique default role.
`name`	`VARCHAR(50)`	4–16 B	Usually a short value such as `user` or `admin`.
`description`	`TEXT`	0–80 B	Optional descriptive text.
Estimated logical row size	**approx. 29–121 B**		Planning model: `row_count x 29-121 B` plus the unique name index and the partial default-role index.

4.1.3 `permissions`

Permissions are normalized as `resource + action`. The generated `name` column gives the application a stable and human-readable permission identifier while keeping the logical source normalized. This is operationally elegant because the generated field can be indexed and queried directly without losing the semantic structure of the permission.

Table 4: `permissions`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`resource`	`VARCHAR(50)`	6–20 B	Examples: `user`, `session`.
`action`	`VARCHAR(50)`	4–16 B	Examples: `read`, `write`, `revoke`.
`name`	`TEXT GENERATED`	12–40 B	Stored generated value `resource:action`.
`description`	`TEXT`	0–80 B	Optional explanatory text.
Estimated logical row size		**approx. 46–180 B**	Planning model: `row_count x 46-180 B` plus the composite uniqueness index and the generated name index.

4.1.4 `role_permissions`

This pivot table is structurally simple but semantically central. It defines which permissions belong to each role. The table is expected to stay small unless the authorization model grows significantly.

Table 5: `role_permissions`: field-level size estimate

Field	SQL type	Typical size	Notes
`role_id`	`UUID`	16 B	Foreign key to `roles`.
`permission_id`	`UUID`	16 B	Foreign key to `permissions`.
Estimated logical row size		**approx. 32 B**	Planning model: `row_count x 32 B` plus the primary key index and the reverse lookup index on permission.

4.1.5 `user_roles`

The `user_roles` table maps accounts to roles and records who granted the role and when. Compared with a plain many-to-many join, this is more valuable operationally because it keeps some assignment provenance.

Table 6: `user_roles`: field-level size estimate

Field	SQL type	Typical size	Notes
`user_id`	`UUID`	16 B	Foreign key to `users`.
`role_id`	`UUID`	16 B	Foreign key to `roles`.
`granted_by`	`UUID`	16 B	Nullable actor who granted the role.
`granted_at`	`TIMESTAMPTZ`	8 B	Assignment timestamp.
Estimated logical row size		**approx. 40–56 B**	Planning model: `row_count x 40-56 B` plus the composite primary key and the role-side lookup index.

4.2 Session and Authentication State

4.2.1 `sessions`

The `sessions` table is one of the most important relations in the entire project. It stores durable session state, refresh-token family lineage, replacement chains, revocation timestamps, compromise markers, device metadata, and the hash of the refresh token itself. It therefore sits at the boundary between security design and operational observability.

This table is also where the project's philosophy becomes very visible: refresh-token handling is not treated as an opaque blob. Instead, the schema is designed so that session state can be inspected, listed, rotated, revoked, and forensically understood.

Table 7: `sessions`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Account owning the session.
`session_family_id`	`UUID`	16 B	Groups rotated sessions into a lineage.
`last_used_at`	`TIMESTAMPTZ`	8 B	Updated on activity.
`expires_at`	`TIMESTAMPTZ`	8 B	Hard expiration boundary.
`created_at`	`TIMESTAMPTZ`	8 B	Session creation timestamp.
`revoked_at`	`TIMESTAMPTZ`	8 B	Present when revoked.
`rotated_at`	`TIMESTAMPTZ`	8 B	Present when rotation occurred.
`compromised_at`	`TIMESTAMPTZ`	8 B	Present after replay or security revocation.
`replaced_by_session_id`	`UUID`	16 B	Points to the next session after refresh rotation.
`ip_address`	`INET`	8–20 B	Client IP recorded for the session.
`device_name`	`VARCHAR(100)`	0–40 B	Short user-facing device label.
`token_hash`	`BYTEA`	32 B	SHA-256 digest of the refresh token.
`user_agent`	`TEXT`	40–120 B	Potentially the heaviest textual column on common rows.
`compromise_reason`	`ENUM`	4 B	Why the session family was marked compromised.
Estimated logical row size		**approx. 220–340 B**	Planning model: `row_count x 220-340 B` plus multiple hot-path indexes for token lookup, active sessions, and family traversal.

From an operational point of view, `sessions` is the clearest example of a table that is not large because of business payload alone, but because the system deliberately keeps security history that many simpler applications would discard.

4.2.2 `two_factor_methods`

The `two_factor_methods` table centralizes TOTP, email OTP, and WebAuthn registrations into one typed registry. The schema avoids invalid payload combinations through a large consistency check, which is preferable to spreading factor logic across several unrelated tables without coordination.

Table 8: `two_factor_methods`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Owner of the factor.
`webauthn_sign_count`	`BIGINT`	8 B	Used only for WebAuthn clone detection.
`last_used_at`	`TIMESTAMPTZ`	8 B	Updated on successful usage.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`updated_at`	`TIMESTAMPTZ`	8 B	Maintained by the trigger.
`method_type`	`ENUM`	4 B	One of `totp`, `email`, or `webauthn`.
`is_primary`	`BOOLEAN`	1 B	Primary factor marker.
`is_verified`	`BOOLEAN`	1 B	Verification state.
`totp_secret`	`TEXT`	48–96 B	Encrypted application-layer payload for TOTP rows only.
`webauthn_credential_id`	`TEXT`	40–120 B	Present only on WebAuthn rows.
`webauthn_public_key`	`TEXT`	120–260 B	Usually the heaviest field on WebAuthn rows.
Estimated logical row size	**email row: approx. 70 B; TOTP row: approx. 130–180 B; WebAuthn row: approx. 260–450 B**		Planning model: total size depends on the factor mix; WebAuthn-heavy deployments make this table materially larger than email- or TOTP-only deployments.

This heterogeneity is precisely why the table deserves explicit documentation. A TOTP row and a WebAuthn row serve the same conceptual role, but they have very different storage and indexing profiles.

4.2.3 `email_2fa_codes`

This relation stores short-lived hashed OTP codes sent during the email-based second-factor challenge. The table is intentionally narrow and ephemeral. Its storage impact comes from write frequency rather than row width.

Table 9: `email_2fa_codes`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Factor owner.
`code_hash`	`BYTEA`	32 B	Hashed OTP code.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`expires_at`	`TIMESTAMPTZ`	8 B	Expiry boundary.
`used_at`	`TIMESTAMPTZ`	8 B	Consumption timestamp.
Estimated logical row size	**approx. 88 B**		Planning model: `row_count x 88 B` plus the user-and-expiry lookup index; operational churn matters more than row width.

4.2.4 `email_verification_tokens`

The verification-token table is more informative than a simple token store because it also records the request IP, user agent, and exact target email being verified. This is valuable for both security review and product logic, especially when email changes are involved.

Table 10: `email_verification_tokens`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Referenced account.
`token_hash`	`BYTEA`	32 B	Stored as a hash, not as plaintext.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`expires_at`	`TIMESTAMPTZ`	8 B	Expiry boundary.
`used_at`	`TIMESTAMPTZ`	8 B	Consumption timestamp.
`request_ip`	`INET`	8–20 B	Origin of the request.
`request_user_agent`	`VARCHAR(255)`	0–80 B	Request metadata, usually shorter than the full limit.
`target_email`	`CITEXT`	20–40 B	Exact email address being verified.
Estimated logical row size		**approx. 116–228 B**	Planning model: `row_count x 116-228 B` plus the token uniqueness index and the active-token partial indexes.

4.2.5 `password_reset_tokens`

This relation is very similar to the email verification table, except that the target email field is not needed. Its storage profile is therefore slightly lighter, but it remains an important operational table because password resets are one of the most security-sensitive flows in the application.

Table 11: `password_reset_tokens`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Referenced account.
`token_hash`	`BYTEA`	32 B	Stored as a hash only.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`expires_at`	`TIMESTAMPTZ`	8 B	Expiry boundary.
`used_at`	`TIMESTAMPTZ`	8 B	Consumption timestamp.
`request_ip`	`INET`	8–20 B	Originating IP address.
`request_user_agent`	`VARCHAR(255)`	0–80 B	Request metadata.
Estimated logical row size		**approx. 96–188 B**	Planning model: `row_count x 96-188 B` plus the token uniqueness index and the active-token partial indexes.

4.2.6 `recovery_codes`

Recovery codes are a backup factor, so the project stores them individually and as hashes. The table is small but very security-sensitive. What matters here is not storage weight but the guarantee that each code can be consumed exactly once and traced individually.

Table 12: `recovery_codes`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Owner of the recovery code.
`created_at`	`TIMESTAMPTZ`	8 B	Creation timestamp.
`expires_at`	`TIMESTAMPTZ`	8 B	Optional expiry.
`used_at`	`TIMESTAMPTZ`	8 B	Consumption timestamp.
`code_position`	`SMALLINT`	2 B	Position within the generated set.
`code_hash`	`BYTEA`	32 B	Hashed recovery code.
Estimated logical row size		**approx. 90 B**	Planning model: `row_count x 90 B` plus the uniqueness indexes on hash and per-user position and the active-code lookup indexes.

4.3 Operational Telemetry

4.3.1 `login_attempts`

The `login_attempts` relation is one of the most operationally important tables in the database. It powers brute-force protection, identifier-based failure windows, IP-based failure windows, lockout logic, and part of the risk-scoring pipeline. Unlike the core identity tables, this one is expected to grow continuously.

Table 13: `login_attempts`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Nullable when the identifier does not map to a known user.
`attempted_at`	`TIMESTAMPTZ`	8 B	Main time dimension.
`attempted_identifier`	`CITEXT`	20–40 B	Email or username supplied by the client.
`was_successful`	`BOOLEAN`	1 B	Outcome flag.
`failure_reason`	`ENUM`	4 B	Present on failed attempts only.
`request_ip`	`INET`	8–20 B	IP used for abuse detection.
`request_user_agent`	`VARCHAR(255)`	0–80 B	Support and forensic context.
Estimated logical row size		**approx. 73–185 B**	Planning model: `row_count x 73-185 B` plus several time-oriented anti-abuse indexes; this is one of the first tables likely to become storage-heavy.

For production capacity planning, `login_attempts` should be treated as one of the first relations to monitor because public authentication traffic can generate writes even when no user successfully authenticates.

4.3.2 `audit_log`

The `audit_log` table is deliberately designed as an append-only, partitioned journal. It records security-relevant actions such as logins, failed logins, password changes, factor enablement or disablement, session compromise events, suspicious logins, and account deletion. Partitioning

by month is the correct design choice because the table is expected to grow continuously and because audit data is naturally retention-driven.

Table 14: `audit_log`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key component together with `created_at`.
`user_id`	`UUID`	16 B	Nullable because some audit events may not map to a durable user record.
`request_id`	`UUID`	16 B	Correlation identifier.
`created_at`	`TIMESTAMPTZ`	8 B	Partition key and event timestamp.
`action`	`ENUM`	4 B	Security action type.
`ip_address`	`INET`	8–20 B	Network origin.
`metadata`	`JSONB`	10–500+ B	Variable payload; this field dominates row width variability.
Estimated logical row size		**approx. 78–580+ B**	Planning model: `row_count x 78-580+ B` per partition, plus partition indexes. The metadata payload and retention period dominate total size.

The audit log is the clearest example of why exact size cannot be known from migrations alone: the `metadata` column can remain tiny for routine events or become significantly larger when detailed event context is retained.

4.3.3 `login_locations`

The `login_locations` table stores successful login context for risk analysis. It is not just a geolocation cache. It captures a stable tuple made of user, country, city, and user agent, then refreshes first-seen and last-seen timestamps over time. This means the table behaves like behavioral memory rather than like a raw append-only event stream.

Table 15: `login_locations`: field-level size estimate

Field	SQL type	Typical size	Notes
`id`	`UUID`	16 B	Primary key.
`user_id`	`UUID`	16 B	Owner of the behavioral entry.
`country`	`TEXT`	8–24 B	Country label from GeoIP resolution.
`city`	`TEXT`	8–40 B	City label from GeoIP resolution.
`user_agent`	`TEXT`	40–120 B	Main discriminating field for device context.
`ip_address`	`INET`	8–20 B	Most recent IP for the tuple.
`latitude`	`DOUBLE PRECISION`	8 B	Optional geographic coordinate.
`longitude`	`DOUBLE PRECISION`	8 B	Optional geographic coordinate.
`last_seen`	`TIMESTAMPTZ`	8 B	Updated on each matching login.
`first_seen`	`TIMESTAMPTZ`	8 B	Creation timestamp of the tuple.
Estimated logical row size		**approx. 120–268 B**	Planning model: `row_count x 120-268 B` plus the uniqueness index used by the upsert path and the last-seen index used by risk lookups.

5 Indexes, Triggers, and SQL-Side Logic

5.1 Index Strategy

The index design in this schema is strongly aligned with authentication hot paths rather than with generic analytics. That is exactly what should happen in a security backend: the indexes serve login lookup, refresh-token validation, per-user session listing, brute-force counting, recent-risk lookup, and audit retrieval.

Table 16: Representative indexes and the queries they accelerate

Relation	Index	Primary purpose
`users`	`users_username_key, users_email_key`	Fast login identifier lookup and uniqueness enforcement.
`sessions`	`idx_sessions_user_active`	List active sessions for the account management UI.
`sessions`	`sessions_token_hash_key`	Refresh-token lookup by hash.
`sessions`	`idx_sessions_family_created`	Walk and revoke refresh-token families.
`two_factor_methods`	`idx_2fa_user_verified`	Resolve verified factors quickly for a given user.
`email_verification_tokens`	`idx_email_verification_tokens_user_active`	Guarantee at most one active verification token per user.
`password_reset_tokens`	`idx_password_reset_tokens_user_active`	Guarantee at most one active password reset token per user.
`login_attempts`	`idx_login_attempts_failed_identifier_time`	Count recent failures by identifier.
`login_attempts`	`idx_login_attempts_failed_ip_time`	Count recent failures by IP.
`audit_log`	`idx_audit_log_created_brin`	Keep range scans over time economical on large partitions.
`login_locations`	`idx_login_locations_upsert`	Enforce uniqueness of the risk-location tuple.

5.2 Triggers and SQL Functions

The project uses a restrained but appropriate amount of SQL-side logic. The database is not overloaded with business rules, but a few functions live there because PostgreSQL is the most reliable place for them.

Table 17: Database-side functions and triggers

SQL object	**Kind**	**Responsibility**
`set_updated_at()`	Function / trigger target	Rewrites `updated_at` before update on mutable tables.
`users_set_updated_at`	Trigger	Keeps `users.updated_at` consistent.
`two_factor_methods_set_updated_at`	Trigger	Keeps `two_factor_methods.updated_at` consistent.
`revoke_session_family(...)`	Function	Revokes every row in a refresh-token family after replay or security action.
`rotate_audit_log_partitions(...)`	Function	Creates future monthly partitions and drops partitions older than retention.
`prevent_audit_log_modification()`	Function / trigger target	Enforces append-only semantics on the audit journal.
`audit_log_append_only`	Trigger	Rejects unauthorized update or delete operations on the audit table.

This is a good balance. Timestamp upkeep, append-only enforcement, and partition rotation all belong naturally in the database. Higher-level business logic remains in the Rust service layer, which keeps the project understandable.

6 Storage Growth and Measurement Strategy

6.1 Which Tables Matter Most for Capacity Planning

From a storage point of view, not all relations deserve the same attention. Small catalog or join tables such as `roles`, `permissions`, and `role_permissions` are architecturally important but operationally cheap. The relations that deserve regular monitoring are those that grow as a function of traffic, security events, or retention policy.

Table 18: Relations that should be monitored first in production

Relation	Growth profile	Why it matters
`audit_log`	Very high	Append-only journal; retention and partition management dominate long-term cost.
`login_attempts`	High	Public traffic can generate continuous writes, including failed traffic.
`sessions`	Medium to high	Refresh rotation and active device history accumulate over time.
`login_locations`	Medium	Successful logins refresh behavioral history and can create new tuples.
`email_verification_tokens` and `password_reset_tokens`	Medium	High operational churn despite relatively small row width.

6.2 Recommended Measurement Query

When a live PostgreSQL instance is available, the repository should be complemented with direct size measurements. The following query is a suitable baseline because it separates heap size, index size, and total relation size:

Listing 1: Suggested PostgreSQL query for relation size measurement

```
SELECT
    relname AS relation_name,
    pg_size_pretty(pg_relation_size(oid)) AS table_size,
    pg_size_pretty(pg_indexes_size(oid)) AS index_size,
    pg_size_pretty(pg_total_relation_size(oid)) AS total_size
FROM pg_class
WHERE relkind = 'r'
  AND relnamespace = 'public'::regnamespace
ORDER BY pg_total_relation_size(oid) DESC;
```

6.3 How to Read the Estimates in This Report

The size tables above should be interpreted as logical payload models. In other words, they answer the question "what kind of row width should an engineer expect from this table?" rather than "what exact number of bytes does PostgreSQL allocate in every case?" For operational planning, that distinction is acceptable and useful.

If an average row width is known or estimated, table footprint can be approximated with the following first-order model:

$$\text{table footprint} \approx (\text{row count} \times \text{average row size}) + \text{index footprint}$$

This is particularly important for `login_attempts` and `audit_log`. Those tables do not become operationally expensive because each row is huge; they become expensive because the system is designed to keep writing to them over time.

III.

API and Rust Codebase

7 Crate Layout and Responsibility Split

The repository uses a layered Rust layout that is well suited to a security-sensitive HTTP backend. The structure is important because the codebase is already large enough that careless growth would quickly produce handlers with embedded SQL, crypto helpers duplicated across modules, or service logic scattered across unrelated files.

Table 19: Main Rust modules and their responsibility

Module	Responsibility
`config`	Loads and validates environment-driven configuration.
`state`	Builds application state, including PostgreSQL, Redis, SMTP, templates, GeoIP, and WebAuthn.
`handlers`	Owns the HTTP layer and route registration.
`middleware`	Provides request IDs, security headers, and Redis-backed rate limiting.
`repositories`	Encapsulates SQL queries and persistence operations.
`services`	Contains business logic, security flows, orchestration, and policy decisions.
`domain`	Declares domain-level types such as users, sessions, audit entries, and tokens.
`utils`	Provides supporting utilities for crypto, JWT, passwords, GeoIP, TOTP, and time.
`src/bin/bench_*`	Benchmark harnesses for HTTP and SQL performance.

This split is one of the strengths of the project. The HTTP layer remains thin, SQL is isolated, and application-wide resources are assembled in one place. That makes the code easier to understand and reduces the risk that one route will evolve in a way that silently bypasses cross-cutting policy.

8 Runtime Bootstrap

The application starts in `src/main.rs`. Startup is deliberately simple and readable:

1. load configuration from the environment;
2. initialize tracing;
3. optionally execute the one-off TOTP key rotation command;
4. build the shared `AppState`;

5. rotate audit partitions at startup;
6. build the Axum router;
7. bind the listener and serve the application.

This is a good bootstrap shape because it keeps one-off maintenance behavior visible and does not bury runtime initialization inside deeply nested abstractions.

8.1 What `AppState` Contains

The `AppState` struct is the operational center of the application. It holds:

- the PostgreSQL pool;
- the Redis pool;
- the SMTP mailer;
- the HTTP client used by external integrations;
- the Tera template engine;
- the loaded configuration;
- the GeoIP database handle;
- the WebAuthn runtime.

This design is clean because expensive or shared resources are initialized once and injected through Axum state rather than being recreated by handlers.

9 HTTP Surface

The router is split into two major branches: public authentication routes under `/auth` and authenticated self-service routes under `/users/me`. That split mirrors the trust model of the application.

9.1 Public Routes

Table 20: Public HTTP routes

Method	Path	Purpose
POST	/auth/register	Account registration.
POST	/auth/login	Primary login with password-based authentication.
POST	/auth/logout	Logout endpoint.
POST	/auth/refresh	Refresh-token rotation and access-token renewal.
GET	/auth/verify-email	Email verification flow.
POST	/auth/forgot-password	Password reset initiation.
POST	/auth/reset-password	Password reset completion.
POST	/auth/two-factor/complete	Generic second-factor completion.
POST	/auth/two-factor/recovery	Recovery-code login path.
POST	/auth/two-factor/email/complete	Email OTP completion.
POST	/auth/two-factor/email/resend	Email OTP resend.
POST	/auth/two-factor/webauthn/start	WebAuthn authentication challenge start.
POST	/auth/two-factor/webauthn/finish	WebAuthn authentication challenge finish.

Public routes are the most exposed surface of the project. They therefore combine several defensive layers: rate limiting, careful error handling, token hashing, lockout logic, risk scoring, and factor challenge flows.

9.2 Protected Self-Service Routes

Table 21: Authenticated self-service routes under `/users/me`

Method	Path	Purpose
GET	/users/me/	Current profile view.
POST	/users/me/reauth	Fresh reauthentication for sensitive actions.
PATCH	/users/me/username	Username change.
PATCH	/users/me/email	Email change.
PATCH	/users/me/password	Password change.
PATCH	/users/me/locale	Locale change.
DELETE	/users/me/	Account deletion.
GET	/users/me/sessions	List active sessions.
DELETE	/users/me/sessions	Revoke all sessions.
DELETE	/users/me/sessions/\{id\}	Revoke one specific session.
POST	/users/me/two-factor/totp/setup	Start TOTP enrollment.
POST	/users/me/two-factor/totp/\{id\}/verify	Verify TOTP enrollment.
DELETE	/users/me/two-factor/totp/\{id\}	Disable TOTP.
POST	/users/me/two-factor/recovery-codes	Regenerate recovery codes.
POST	/users/me/two-factor/recovery-codes/use	Consume a recovery code.
POST	/users/me/two-factor/email/setup	Start email OTP enrollment.
POST	/users/me/two-factor/email/send	Send email OTP code.
POST	/users/me/two-factor/email/\{id\}/verify	Verify email OTP enrollment.
DELETE	/users/me/two-factor/email/\{id\}	Disable email OTP.
POST	/users/me/two-factor/webauthn/register/start	Start WebAuthn registration.
POST	/users/me/two-factor/webauthn/register/finish	Finish WebAuthn registration.
DELETE	/users/me/two-factor/webauthn/\{id\}	Disable a WebAuthn credential.

The self-service surface is already fairly rich. That is one of the reasons why a layered architecture matters here: account changes, session changes, second-factor changes, and sensitive reauthentication all require different combinations of repository access, validation, and audit logging.

10 Internal Request Flow

Although each route has its own details, the application follows one stable request pattern:

1. the request enters the Axum router;
2. global middleware attaches a request ID, security headers, and rate limiting;
3. the handler parses and validates HTTP-facing input;
4. the service layer applies the actual business and security rules;

5. repositories execute the necessary SQL operations;
6. side effects such as mail, audit writes, or token issuance are performed;
7. the response is returned with a stable success or error shape.

This pattern is important because it keeps route code readable. A maintainer can start from the router, move into the handler, then into the service, and finally into the repository. That mental model is easy to teach and easy to preserve during future refactors.

11 Functional Domains

11.1 Authentication and Account Lifecycle

The authentication stack covers registration, email verification, password-based login, password reset, and logout or refresh flows. Importantly, those flows are not isolated from the rest of the project. Registration interacts with roles and default-role assignment; login interacts with sessions, login telemetry, factor challenge state, risk scoring, and audit logging; password reset interacts with one-time token storage and with downstream password-change security actions.

11.2 Sessions and Token Rotation

Session management is more mature than a typical tutorial implementation. Sessions are durable, listable, revocable, and organized into families. Refresh-token rotation does not merely mint a new token; it also modifies the persistent lineage that makes replay detection and family-wide compromise handling possible.

11.3 Second Factors

The project supports three families of second factors: TOTP, email OTP, and WebAuthn. This is valuable because it allows the system to adapt to different trust and UX models. TOTP provides an offline authenticator-app factor, email OTP provides a fallback or lower-security but convenient factor, and WebAuthn provides phishing-resistant credentials. The existence of recovery codes completes that design by providing a controlled recovery path when the primary factor is unavailable.

11.4 Risk, Audit, and User Notification

The project does not stop at authentication success or failure. It also records login attempts, tracks login locations, computes risk signals, writes audit entries, and can send mail when certain thresholds are crossed. This is one of the reasons the repository deserves to be described as a security backend rather than as a simple “login API.”

12 How to Add a New Feature

As the repository grows, consistency matters more than speed of implementation. The safest order for adding a new feature is therefore architectural rather than opportunistic.

12.1 Step 1: Define the Functional and Security Scope

Before writing code, define what the feature changes in terms of user-visible behavior, state transitions, and security requirements. For example, a new "trusted device" feature would interact with sessions, login locations, risk scoring, and probably the audit log. That means the feature should not be approached as a single new handler.

12.2 Step 2: Update the Schema if Persistent State Changes

If the feature requires new durable state, the migration should come first. This keeps the repository honest: the schema defines what the application is allowed to store, and the rest of the stack grows on top of that contract.

12.3 Step 3: Extend Repositories Before Extending Handlers

Once the schema exists, add or update repository functions for the relevant queries. This preserves the layered design and prevents handlers from growing direct SQL responsibilities.

12.4 Step 4: Add the Service-Level Policy

The service layer is where rules should live: revocation policy, factor validation policy, mail-sending triggers, or audit semantics. This keeps the HTTP layer thin and makes behavior easier to test outside of Axum.

12.5 Step 5: Expose the Feature Through Routes

Only after the domain, schema, and service behavior are clear should the handler and route be added. Doing it in that order reduces architectural drift and makes code review easier because each layer has a clear purpose.

12.6 Step 6: Add Tests, Benchmarks, and Documentation

If the feature touches a hot authentication path, benchmark coverage should be considered as part of the implementation rather than as an optional afterthought. If it changes runtime configuration or deployment expectations, the deployment documentation should be updated in the same change set.

13 Benchmark Surface and Performance Interpretation

13.1 What Is Benchmarked in the Repository

The repository already includes two dedicated benchmark binaries:

- `src/bin/bench_http.rs`, which runs HTTP scenarios against a real Axum server backed by PostgreSQL and Redis;
- `src/bin/bench_sql.rs`, which runs query-level measurements and captures one `EXPLAIN ANALYZE` plan per scenario.

This is a very useful setup because it separates application-level latency from query-level latency. In practice, that distinction matters a lot: password verification can dominate request latency even when the underlying queries are already fast.

13.2 HTTP Benchmark Coverage

Table 22: HTTP benchmark scenarios and their main cost center

Scenario	Expected latency class	Dominant cost center
register	High	Password hashing, account creation, and mail-related work.
login_success_email	High	Password verification plus session creation and logging.
login_success_username	High	Same class as email login.
login_wrong_password	High	Password verification still dominates even on failure.
forgot_password	Low to moderate	Token issuance and mail flow preparation.
refresh	Very low	Session hash lookup, rotation, and token issuance.
get_profile	Very low	Authenticated read with light database work.
list_sessions	Very low	Hot indexed query over the session table.
reauthenticate	High	Password verification for a fresh-trust operation.
change_locale	Low	Small authenticated update.
email_2fa_challenge	High	Primary authentication plus challenge issuance.
email_2fa_complete	Moderate	Code verification and follow-up state changes.
totp_challenge	High	Primary authentication plus TOTP challenge path.
totp_complete	Moderate	TOTP verification plus session finalization.

Even without freezing one specific benchmark run into the report, the performance shape is already clear. Password-based and reauthentication flows sit in the expensive class because Argon2 is intentionally costly. In contrast, refresh, profile, session listing, and small account updates are lightweight because the underlying SQL paths are well indexed and the application is not doing heavy cryptographic work there.

13.3 SQL Benchmark Coverage

Table 23: SQL benchmark scenarios covered by the repository

Scenario	Runtime meaning
find_by_identifier_email	Hot login lookup by email.
find_by_identifier_username	Hot login lookup by username.
find_validation_by_id	Session validation on authenticated requests.
find_by_token_hash	Refresh-token lookup.
find_active_by_user	Session management list view.
find_recent_for_risk	Recent login-location lookup for risk scoring.
count_recent_failures_by_identifier	Brute-force counting by login identifier.
count_recent_failures_by_ip	Brute-force counting by source IP.
count_consecutive_failures_by_user	Lockout-oriented consecutive-failure counting.
login_location_upsert	Steady-state successful-login location update path.

This SQL benchmark set is well chosen because it covers the exact queries that define the perceived responsiveness and the security behavior of the system. The project is not benchmarking arbitrary reads; it is benchmarking the paths that actually matter.

13.4 Main Performance Interpretation

The main performance conclusion of the current codebase is not that "everything must become faster." The more precise conclusion is that the system already separates into two classes of work:

- cryptographically expensive flows, where latency is dominated by Argon2 or factor verification and where that cost is mostly intentional;
- I/O-bound but well-indexed flows, where latency remains low because the SQL access pattern is healthy.

That distinction matters when tuning the project. If a login path is expensive, the first question should be whether the cost is intentional cryptography or avoidable persistence overhead. The current repository strongly suggests that the former is often the dominant explanation.

IV.

Infrastructure and Deployment

14 Deployment Topology

The deployment model documented in the repository separates the system into two servers:

- an application server;
- a data server.

This split is operationally sound. It separates HTTP-facing concerns from durable state concerns and gives the deployment a clear shape that can be reasoned about, documented, and secured.

Table 24: Server responsibilities in the current deployment model

Server	Main assets	Operational role
Application server	Runtime API container, Nginx, GeoIP file, runtime configuration, local `pass` store	Serves HTTP traffic, exposes the public service, and consumes PostgreSQL and Redis as external dependencies.
Data server	PostgreSQL, Redis, runtime configuration, local `pass` store, migration image execution	Holds durable service state and runs the database migration step.

15 Docker Image Model

The repository builds two distinct Docker images from the same `Dockerfile`:

- a runtime image for the API;
- a migrations image containing `sqlx-cli` and the migration files.

That split is an architectural strength. Database migrations are not just "something the app does on boot." They are their own operational concern and deserve their own build target, their own release artifact, and their own execution step.

15.1 Runtime Image

The runtime image is intentionally minimal. It ships the compiled binary, the mail templates, CA certificates, and a non-root application user. The container is read-only at runtime except for a small temporary filesystem, and Linux capabilities are dropped. These are meaningful hardening choices for a public-facing service.

15.2 Migrations Image

The migrations image exists for a different purpose: it allows a server to apply schema changes without cloning the repository. That is important in the documented deployment model, where the server is expected to receive images and local deployment files, not the whole source tree.

16 Secrets and Encrypted Service Roots

16.1 Local Secret Management with `pass`

The repository no longer relies on an external secret manager. Instead, each server keeps a local encrypted `pass` store backed by GPG. This is a pragmatic compromise: secrets are not left as plaintext environment files on disk, but the operational model remains understandable and self-hosted.

In the current documentation:

- application-side secrets are stored under a hierarchy such as `rust-api/app/...`;
- data-side secrets are stored under a hierarchy such as `rust-api/data/...`.

At startup, secrets are exported from `pass` into the shell environment and then consumed by Docker Compose.

16.2 Encrypted Service Roots with `cryptsetup`

The deployment guides also recommend encrypting:

- `/srv/rust-api`;
- `/srv/rust-api-data`

with LUKS volumes managed through `cryptsetup`. This is operationally coherent with the local-secret model:

- `pass` protects individual secret values;
- encrypted service roots protect the broader server filesystem at rest.

The combination is particularly suitable for self-hosted infrastructure where the operator wants local control without running a full secret-management platform.

17 Application Server Procedure

The application server procedure documented in `docs/deploy/app/setup.md` is well ordered and should be preserved as such:

1. prepare the encrypted service root if LUKS is used;
2. create `/srv/rust-api/env/runtime.env`;
3. populate application-side secrets in `pass`;
4. prepare the GeoIP database file if risk scoring requires it;
5. create `/srv/rust-api/compose/docker-compose.yml`;
6. install and configure Nginx;
7. export secrets and start the application container.

This order is not arbitrary. The application should not be started before the reverse proxy, runtime configuration, and local secret store are ready. Likewise, GeoIP data should exist before startup if the configuration is strict about it.

17.1 Why GeoIP Lives on the Application Server

GeoIP is not a service of its own. It is a data file consumed locally by the Rust process. That is why it belongs on the application server, mounted into the runtime container, rather than on the data server. This is a small but important example of the deployment model remaining faithful to the actual runtime dependency graph.

18 Data Server Procedure

The data server procedure documented in `docs/deploy/data/setup.md` is similarly structured:

1. prepare the encrypted service root if LUKS is used;
2. create `/srv/rust-api-data/env/runtime.env`;
3. populate PostgreSQL and Redis secrets in `pass`;
4. create `/srv/rust-api-data/compose/docker-compose.yml`;
5. start PostgreSQL and Redis;
6. initialize PostgreSQL roles and the application database;
7. validate Redis protection and connectivity;
8. run the migrations image;
9. construct the final application-side `DATABASE_URL` and `REDIS_URL`.

This deployment order is particularly important because the application server depends on data services that already exist and are already migrated. The runtime application image is not supposed to bootstrap the database by itself.

18.1 Why PostgreSQL and Redis Share a Data Server Here

The documented topology treats PostgreSQL and Redis as colocated on the data server. That is a reasonable trade-off in a self-hosted environment: Redis stays close to the application's abuse-control and session-support needs, while PostgreSQL remains the durable source of truth. The two services have different lifecycles, but they share the same broad operational perimeter of "stateful backend dependencies."

19 Release Pipeline and CI Workflows

The repository currently documents and uses four main GitHub Actions workflows.

Table 25: Main GitHub Actions workflows

Workflow	Trigger	Role
`code-checks.yml`	Pull request to `main`	Runs formatting, Clippy, and tests for the Rust codebase.
`docker-checks.yml`	Pull request to `main`	Runs Docker-oriented checks such as image build, Dockerfile linting, and container security scanning.
`security.yml`	Pull request to `main`	Runs dependency and repository security checks such as `cargo audit`, `cargo deny`, `gitleaks`, and SBOM generation.
`docker-publish.yml`	Tag push `v*`	Builds and publishes the runtime and migrations images to GHCR. Publication is allowed only when the tagged commit belongs to `main`.

19.1 Release Semantics

Releases are tag-driven. A version tag such as `v1.0.0` does not create a new commit; it simply points to an existing commit and triggers image publication. That is a clean release model because it keeps source history and release identity separate. The published artifacts are:

- `ghcr.io/<owner>/rust-api:<tag>`;
- `ghcr.io/<owner>/rust-api-migrations:<tag>`.

This means the servers do not need the repository itself. They only need:

- local deployment files created from the repository documentation;
- the published images;
- the local secret store and encrypted service root.

20 Why the Deployment Model Is Coherent

The main strength of the deployment design is not that it is extremely complex. On the contrary, its strength is that it is explicit and proportionate:

- the application server serves the API and nothing else;
- the data server owns PostgreSQL, Redis, and migrations;
- the runtime and migrations are separate images;

- secrets are local and encrypted rather than outsourced;
- service roots can be encrypted without changing Compose paths;
- image release is centralized in GitHub Actions and consumption is local.

For a self-hosted project, this is a strong operational baseline. It avoids both extremes: a completely ad hoc deployment and an overly abstract platform that would be heavier than the project itself.

V.

Monitoring and Observability

21 Observability Objectives

Once a backend moves beyond trivial CRUD behavior, operational visibility becomes part of the design rather than a postscript. This is especially true for an authentication-oriented system. Operators do not only need to know whether the process is running; they also need to understand whether PostgreSQL is emitting warnings, whether Redis is behaving correctly, whether migrations failed, whether suspicious login activity is rising, and whether the reverse proxy and runtime container are producing error patterns that deserve intervention.

The repository now documents a monitoring model built around Grafana, Loki, and Alloy, with PostgreSQL remaining the durable source for structured audit and login-attempt data. This is an important distinction. The project does not treat every operational signal as an unstructured log line. Instead, it separates two categories of evidence:

- container and service logs, which are best handled by a log aggregation system;
- structured security telemetry, which remains in PostgreSQL and can later be queried from Grafana through a read-only datasource.

This split is coherent with the rest of the project. Append-only audit records, login attempts, and derived security signals already have explicit relational structure and should keep it. In contrast, runtime messages from PostgreSQL, Redis, Grafana, Loki, Alloy, or the application process itself belong naturally to a log stream.

22 Monitoring Topology

In the currently documented model, the observability stack is hosted on the data server. This means that the same host may run:

- PostgreSQL and Redis as stateful backend services;
- Loki as the log store;
- Grafana as the visualization layer;
- Alloy as the local agent reading Docker container logs.

This topology is not the only possible one, but it is a reasonable self-hosted compromise. It keeps the monitoring stack on a server that is already private, already stateful, and already expected to remain under the operator's control.

Table 26: Monitoring components and their roles

Component	Placement	Operational role
Alloy	Data server	Reads Docker logs locally, applies labels, and forwards streams to Loki.
Loki	Data server	Stores and indexes container logs for exploration and dashboard queries.
Grafana	Data server	Exposes the monitoring UI, dashboards, and ad hoc exploration.
PostgreSQL datasource	Data server / Grafana	Allows Grafana to query structured tables such as `audit_log` and `login_attempts` through a dedicated read-only role.

22.1 Why Loki Does Not Replace `audit_log`

An important architectural decision is that Loki is not used as a substitute for the SQL audit ledger. This is the correct choice. The `audit_log` table is append-only, partitioned, queryable with relational semantics, and indexed according to the security questions the system expects to answer. Replacing that with pure log search would weaken the model rather than strengthen it.

Loki should instead be understood as the runtime-observation layer. It answers questions such as:

- What did PostgreSQL log during the last hour?
- Is Redis emitting errors or warnings?
- Did Grafana or Alloy fail to start correctly?
- What happened in the migration container during the last execution?

The audit schema answers a different class of questions:

- How many suspicious logins were recorded today?
- Which requests triggered session-family revocation?
- Which users accumulated repeated failed logins?
- Which security actions occurred for one correlation identifier?

Keeping both layers separate produces a cleaner and more trustworthy operational model.

23 Log Collection Model

The deployed Alloy configuration uses Docker discovery and labels each stream with service-related metadata before forwarding it to Loki. This means the operator can search logs by container name, Docker Compose service, project, stream, and the manually attached role labels used in the repository configuration.

Conceptually, the pipeline is:

1. Docker produces stdout/stderr log streams for each container.
2. Alloy discovers those containers on the local host.
3. Alloy attaches stable labels such as `service`, `container`, and `server_role`.
4. Loki ingests and stores the resulting streams.
5. Grafana queries Loki and renders dashboards or ad hoc views.

This is a light architecture by modern observability standards, but it is proportionate to the project. It avoids the complexity of a heavyweight search cluster while still giving centralized log search and dashboards.

23.1 Expected Early Log Coverage

At the current stage of the repository, the monitoring stack is primarily designed to centralize:

- PostgreSQL logs;
- Redis logs;
- Grafana, Loki, and Alloy logs;
- migration-container logs when the migrations image is executed on the data server.

The application server can later join the same stack by running its own Alloy agent and forwarding logs to the same Loki instance. This staged rollout is sensible because it keeps the first monitoring milestone small: observe the private backend host first, then widen the scope to the public application server.

24 Dashboards and Operational Reading

The repository now includes provisioned Grafana dashboards. Their purpose is not to be a full observability platform on day one, but to provide an immediately useful baseline once the monitoring stack starts.

Table 27: Provisioned dashboard baseline

Dashboard	Primary source	Purpose
`Data Server Overview`	Loki	Gives a first operational picture of log volume, recent errors, and recent container output across the main data-server services.
`PostgreSQL and Redis Logs`	Loki	Focuses on the two stateful backend dependencies and exposes their log streams directly.
`Observability Services`	Loki	Observes the monitoring stack itself, which is important because an unobserved monitoring stack quickly becomes misleading.

These dashboards are intentionally log-centric. They answer the most basic and most useful first questions:

- Are services producing logs at all?
- Are there obvious recent errors?

- Can an operator immediately inspect PostgreSQL or Redis output?
- Is the monitoring stack itself healthy enough to trust?

This baseline is appropriate. Operational monitoring should begin with trustworthy signal acquisition before expanding into more sophisticated panels.

24.1 The Next Layer: SQL Dashboards

After the PostgreSQL read-only datasource is configured in Grafana, a second class of dashboards becomes possible. These dashboards should focus on structured tables rather than on free-form logs. Typical candidates include:

- failed logins by hour from `login_attempts`;
- suspicious logins over time from `audit_log`;
- session revocations and replay detections from `audit_log`;
- recent behavioral location changes from `login_locations`.

From an architectural perspective, this is where the project becomes particularly strong: Grafana can act as a single observation interface while still respecting the difference between logs and relational evidence.

25 Security of the Monitoring Stack

Monitoring systems are often deployed carelessly because they are considered "internal." That would be a mistake here. The repository documentation adopts a healthier posture:

- Grafana should remain private unless intentionally published behind a secured reverse proxy;
- Loki should remain private and should not be exposed broadly;
- Alloy's debug UI should remain local or at most private-network only;
- the PostgreSQL datasource should use a dedicated read-only role rather than reusing the application or migration role.

This is a coherent security stance. A monitoring stack has visibility into service behavior, identifiers, and potentially sensitive failure modes. It should therefore be treated as an operationally privileged subsystem, not as a disposable convenience service.

26 Why the Monitoring Model Fits the Project

The monitoring design is a good fit for the overall character of the repository for three reasons.

First, it remains self-hosted and proportionate. The project does not need a large multi-node observability platform to become observable. Grafana, Loki, and Alloy are sufficient to give useful visibility at the current scale.

Second, it preserves semantic clarity. Runtime logs go to Loki. Security facts remain in PostgreSQL. This avoids the common mistake of pushing every kind of operational truth into one tool regardless of structure.

Third, it integrates naturally with the documented deployment model. The stack can be deployed on the data server using the same documentation-driven workflow as the rest of the infrastructure, with local configuration files, a local `pass` store, and predictable filesystem paths under `/srv/rust-api-data`.

VI.

Conclusion

27 Final Assessment

The Rust API project already has the characteristics of a serious backend system. Its PostgreSQL schema is security-aware and structurally coherent. Its Rust codebase is layered in a way that keeps the HTTP layer readable and isolates persistence logic properly. Its benchmark harnesses show that the project has moved beyond intuition and into explicit measurement. Finally, its deployment model is no longer an informal script collection, but a documented architecture split across application and data responsibilities.

The most important conclusion of this report is that the project is consistent across layers. The database does not contradict the service logic. The service logic does not fight the deployment model. The deployment model does not pretend that migrations, secrets, reverse proxying, and runtime serving are the same concern. This coherence is the main technical asset of the repository today.

Future work should therefore preserve that quality. New features should continue to be introduced through migrations, repositories, services, handlers, tests, benchmark coverage, and deployment documentation in that order. If this discipline is maintained, the project can grow without losing the clarity that currently makes it understandable.